

Data Analysis using Python

07 – Data Preparation

Contents

Why clean and prepare data

Missing Values

What is Data Preparation and Cleaning

Data preparation involves transforming raw data into a usable format.

Data cleaning focuses on removing inaccuracies, inconsistencies, and errors in the data.

Importance of Data Preparation and Cleaning

Ensures Data Quality

Raw data often contains

Missing values.

Duplicate records.

Outliers or inconsistencies.

Cleaning ensures data is accurate, consistent, and complete.

Improves Analysis Accuracy

Reliable data leads to

Meaningful insights.

Accurate predictions and conclusions.

Prevents skewed results caused by incorrect or incomplete data.

Importance of Data Preparation and Cleaning

Saves Time in the Long Run

Well-prepared data reduces
Rework during analysis.
Debugging time for errors in data-driven applications.

Optimizes Machine Learning Performance

Clean, well-structured data improves
Model training efficiency.
Prediction accuracy in machine learning workflows.

Supports Business Decisions

High-quality data is essential for
Strategic planning.
Monitoring KPIs.
Driving evidence-based decisions.

Common Data Cleaning Tasks

Handling Missing Values

Impute missing data or drop incomplete records.

Removing Duplicates

Identify and eliminate duplicate rows.

Correcting Inconsistent Data

Standardize formats (e.g., date/time, currencies).

Outlier Treatment

Detect and decide how to handle extreme values.

Data Type Conversion

Ensure correct data types for analysis (e.g., integers for numerical data).

dropna

A Pandas method used to remove rows or columns with missing values (NaN).

Essential for cleaning datasets where missing values impact analysis.

Why Use dropna?

Ensures Data Consistency:

Eliminates incomplete rows or columns that may distort analysis.

Prepares Data for Analysis:

Many algorithms and visualizations require non-missing values.

Customizable:

Offers flexibility in dropping rows/columns based on thresholds.

dropna

Syntax

DataFrame.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False)

Key Parameters

axis

0 (default): Drop rows with NaN.

1 Drop columns with NaN.

how

any (default): Drop rows/columns with any NaN.

all Drop rows/columns only if all values are NaN.

thresh Minimum non-NA values required to retain row/column.

subset Specific columns to check for NaN.

inplace Modify the DataFrame in place or return a new one.

dropna

Example 1: Dropping Rows with Missing Values

```
import pandas as pd
```

Sample DataFrame

```
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],  
        'Age': [25, None, 30, 28],  
        'Salary': [50000, 60000, None, 70000]}
```

```
df = pd.DataFrame(data)
```

Drop rows with any NaN values

```
cleaned_df = df.dropna()
```

```
print(cleaned_df)
```

dropna

```
# Example 2: Dropping Columns with Missing Values
```

```
# Drop columns with any NaN values
```

```
cleaned_df_cols = df.dropna(axis=1)
```

```
print(cleaned_df_cols)
```

```
# Example 3: Using thresh for Flexible Cleaning
```

```
# Drop rows with fewer than 2 non-NA values
```

```
flexible_clean = df.dropna(thresh=2)
```

```
print(flexible_clean)
```

Missing Values

Why Handle Missing Values?

Impact of Missing Values:

Can distort analysis and lead to inaccurate results.

Many algorithms do not support missing data.

Goal:

Detect, analyze, and handle missing values effectively.

Missing Values

Detecting Missing Values

Methods:

`isna()` **or** `isnull()`

Detects NaN values and returns a DataFrame of True/False.

`notna()` **or** `notnull()`

Detects non-missing values.

`sum()`

Count missing values per column or row.

Missing Values

Detecting Missing Values

```
import pandas as pd
```

Sample DataFrame

```
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],  
        'Age': [25, None, 30, 28],  
        'Salary': [50000, 60000, None, 70000]}  
df = pd.DataFrame(data)
```

Detect missing values

```
missing = df.isna()
```

Count missing values per column

```
missing_count = df.isna().sum()
```

```
print(missing)
```

```
print(missing_count)
```

Missing Values

Handling Missing Values

Dropping Missing Values (**dropna**)

Removes rows/columns with missing data.

```
# 1. Dropping Missing Values (dropna)  
cleaned_df = df.dropna()
```

Missing Values

Filling Missing Values (fillna)

Replace missing values with

A specific value.

Statistical measures (e.g., mean, median, mode).

2. Filling Missing Values (fillna)

Fill with a constant value

```
df['Age'] = df['Age'].fillna(0)
```

Fill with the column mean

```
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
```

Missing Values

Interpolation

Predict missing values based on existing data

```
# Interpolate missing values  
df['Age'] = df['Age'].interpolate()
```

Dealing With Duplicates

Duplicate rows are records with identical values across all or specific columns.

Commonly introduced through

- Data collection errors.

- Merging datasets.

- Repeated observations.

Why Remove Duplicates?

- Ensure Data Quality

 - Duplicates distort analysis and skew results.

- Optimize Storage

 - Reduces dataset size by removing redundant rows.

- Improve Accuracy

 - Ensures reliable insights from unique records.

Dealing With Duplicates

Detecting Duplicates

df.duplicated()

Returns a Boolean Series indicating duplicate rows.

```
import pandas as pd

# Sample DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie', 'Alice'],
        'Age': [25, 30, 35, 25],
        'Salary': [50000, 60000, 70000, 50000]}
df = pd.DataFrame(data)

# Detect duplicates
duplicates = df.duplicated()
```

Dealing With Duplicates

Removing Duplicates

`drop_duplicates()`

Syntax

```
DataFrame.drop_duplicates(subset=None, keep='first', inplace=False)
```

Key Parameters

subset: Specifies columns to consider for duplicates. Default is all columns.

keep:

first (default) Keeps the first occurrence.

last Keeps the last occurrence.

False Removes all duplicates.

inplace: Whether to modify the original DataFrame.

Dealing With Duplicates

Example 1. Removing All Duplicates

```
unique_df = df.drop_duplicates()  
print(unique_df)
```

Example 2. Considering Specific Columns

Remove duplicates based only on 'Name'

```
unique_names = df.drop_duplicates(subset=['Name'])  
print(unique_names)
```

Example 3. Retaining the Last Occurrence

Keep the last occurrence of duplicates

```
last_occurrence = df.drop_duplicates(keep='last')  
print(last_occurrence)
```

Other Considerations

Preparing Data is a huge topic.

Other more advanced techniques may involve

- categorization

- label and one hot encoding

- feature scaling

- normalization

- date and string manipulation

- internationalization

- the application of advanced statistical processes and methods

